

Struts2-002

漏洞介绍

Struts2-002是一个 **XSS** 漏洞，该漏洞发生在 `<s:url>` 和 `<s:a>` 标签中，当标签的属性 `includeParams=all` 时，即可触发该漏洞。

漏洞版本

Struts 2.0.0 - Struts 2.1.8.1 。

官方通告: <https://cwiki.apache.org/confluence/display/WW/S2-002>

环境搭建

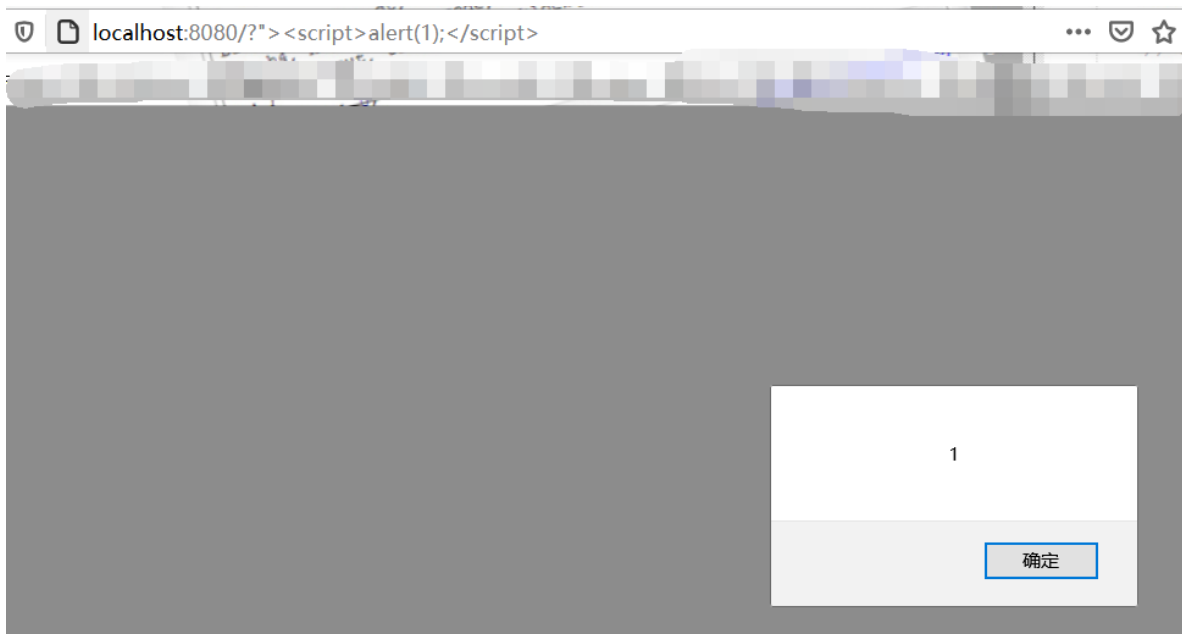
这里的环境我们使用上一个的 S2-001 Demo

修改一下 `index.jsp`

```
1 <!DOCTYPE html PUBLIC
2 "-//W3C//DTD XHTML 1.1 Transitional//EN"
3 "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
4
5 <%@taglib prefix="s" uri="/struts-tags" %>
6
7 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
8
9 <body>
10 <a href="<s:url action="login" includeParams="all" ></s:url">Struts2-
   xss</a>
11 </body>
12 </html>
13
```

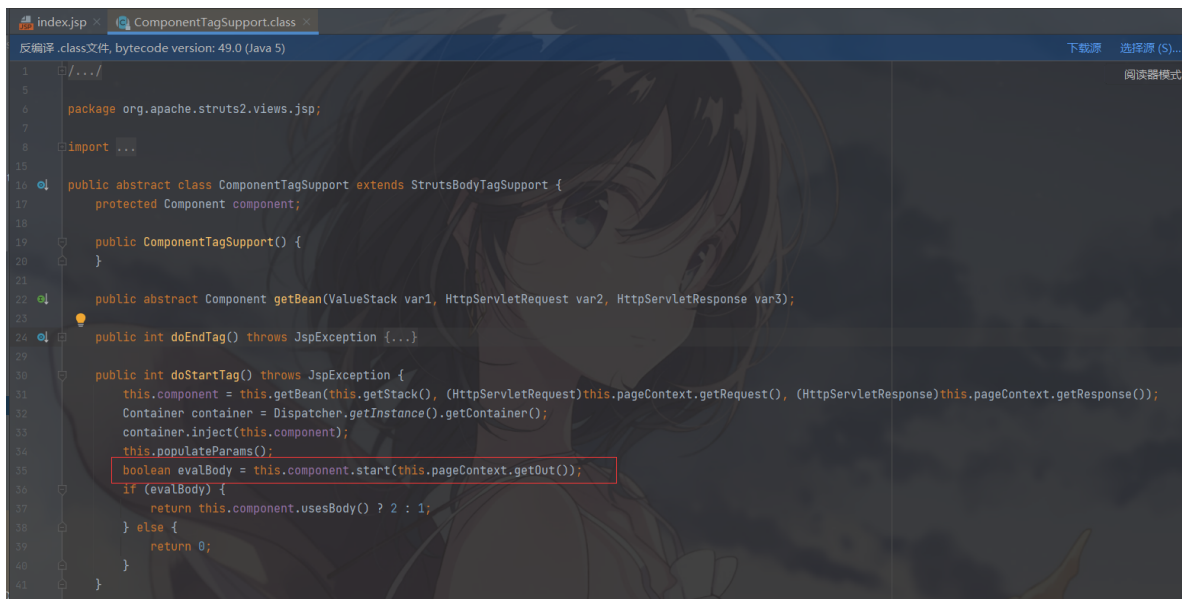
复现

poc: ?"><script>alert(1);</script>



漏洞分析

这里介绍一下当程序开始解析 **Struts** 标签时，会调用 **org.apache.struts2.views.jsp.ComponentTagSupport:doStartTag()** 方法。



由于我们这里处理的是 **s:url** 标签，所以这里用来处理标签的组件 **this.component** 为 **org.apache.struts2.components.URL** 类对象。我们跟进 **URL:start()** 方法。注意 `index.jsp` 中 `includeParams="all"`

```

79
80     try {
81         String includeParams = this.urlIncludeParams != null ? this.urlIncludeParams.toLowerCase() : "get";
82         if (this.includeParams != null) {
83             includeParams = this.findString(this.includeParams);
84         }
85
86         if ("none".equalsIgnoreCase(includeParams)) {
87             this.mergeRequestParameters(this.value, this.parameters, Collections.EMPTY_MAP);
88         } else if ("all".equalsIgnoreCase(includeParams)) {
89             this.mergeRequestParameters(this.value, this.parameters, this.req.getParameterMap());
90             this.includeGetParameters();
91             this.includeExtraParameters();
92         } else if (!"get".equalsIgnoreCase(includeParams) && (includeParams != null || this.value != null || this.act
93             if (includeParams != null) {
94                 LOG.warn("Unknown value for includeParams parameter to URL tag: " + includeParams);
95             }
96         } else {
97             this.includeGetParameters();
98             this.includeExtraParameters();
99         }
100     } catch (Exception var4) {
101         LOG.warn("Unable to put request parameters (" + this.req.getQueryString() + ") into parameter map.", var4)
102     }

```

在 `URL:start()` 方法中，我们看到当 `includeParams=all` 时，会调用 `mergeRequestParameters` 方法。

我们跟进这个方法

```

271     protected void mergeRequestParameters(String value, Map parameters, Map contextParameters) {
272         Map mergedParams = new LinkedHashMap(contextParameters);
273         if (value != null && value.trim().length() > 0 && value.indexOf("?") > 0) {
274             new LinkedHashMap();
275             String queryString = value.substring(value.indexOf("?") + 1);
276             mergedParams = UrlHelper.parseQueryString(queryString);
277             Iterator iterator = contextParameters.entrySet().iterator();
278
279             while(iterator.hasNext()) {...}
280         }
281
282         Iterator iterator = ((Map)mergedParams).entrySet().iterator();
283
284         while(iterator.hasNext()) {
285             Entry entry = (Entry)iterator.next();
286             Object key = entry.getKey();
287             if (!parameters.containsKey(key)) {
288                 parameters.put(key, entry.getValue());
289             }
290         }
291     }
292
293 }
294
295
296
297
298

```

在 `mergeRequestParameters` 方法中，程序会将 `this.req.getParameterMap()` 获得的键值对数据存入 `this.parameters` 属性。

```

89     this.mergeRequestParameters(this.value, this.parameters, this.req.getParameterMap());
90     this.includeGetParameters();
91     this.includeExtraParameters();

```

在 `mergeRequestParameters` 方法之后，依次是 `includeGetParameters`、`includeExtraParameters` 方法。 `includeGetParameters` 方法主要是将 `HttpRequest.getQueryString()` 的数据存入 `this.parameters` 属性。

```

114     private void includeGetParameters() {
115         if (!Dispatcher.getInstance().isPortletSupportActive() || !PortletActionContext.isPortletRequest()) {
116             String query = this.extractQueryString();
117             this.mergeRequestParameters(this.value, this.parameters, UrlHelper.parseQueryString(query));
118         }
119     }
120 }

```

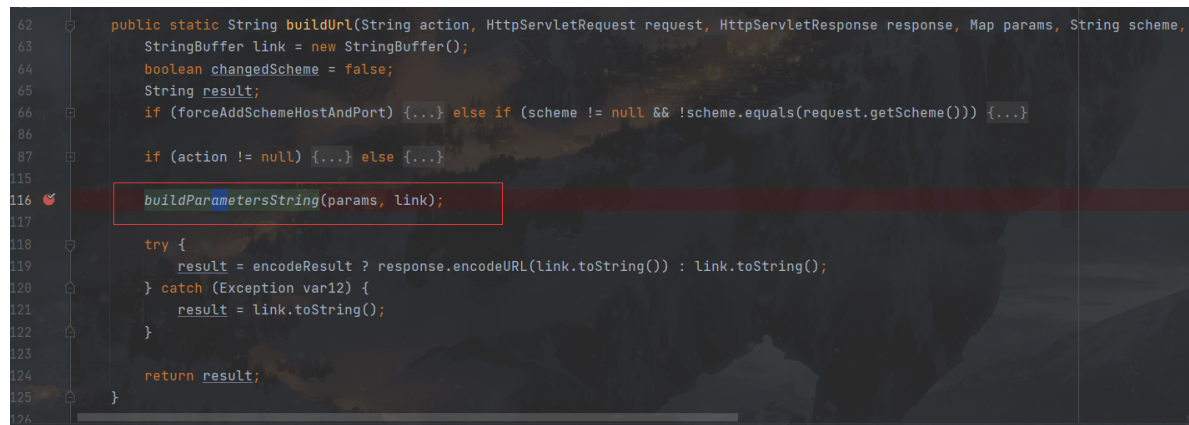
而 `includeExtraParameters` 方法则用来存入其它数据。

```

107     private void includeExtraParameters() {
108         if (this.extraParameterProvider != null) {
109             this.mergeRequestParameters(this.value, this.parameters, this.extraParameterProvider.getExtraParameters());
110         }
111     }
112 }

```

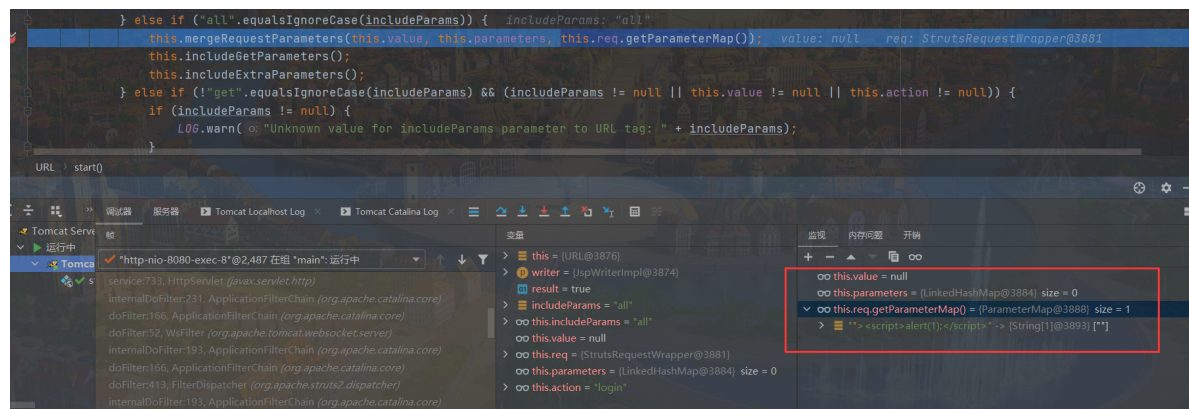
最后会由 `URLHelper` 类处理 `this.parameters` 数据（将其拼接在URL中）并返回给用户，这也导致最终的 **XSS 漏洞**。



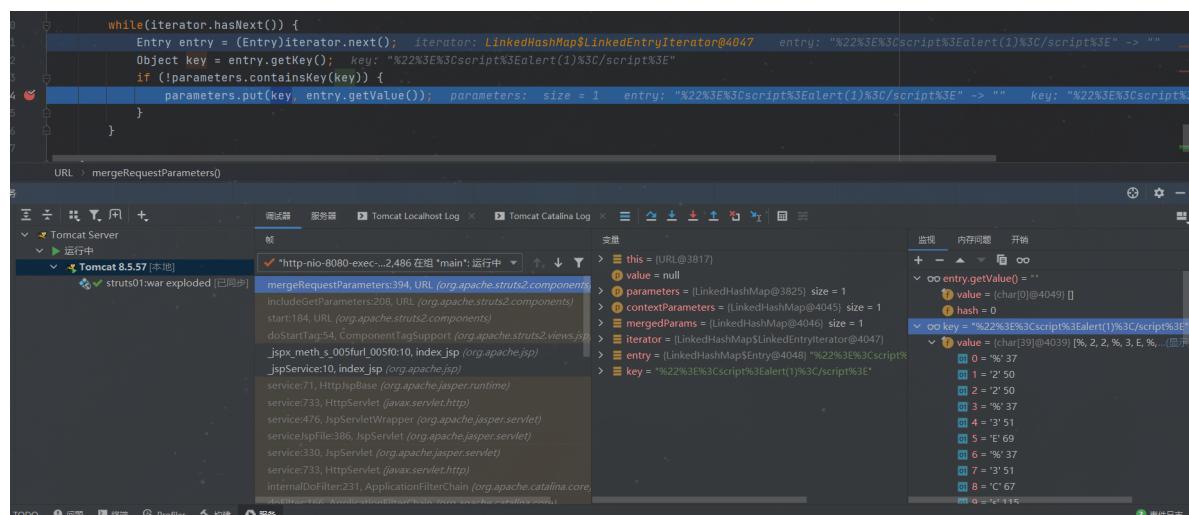
动态调试

动态调试是为了更加理解，毕竟自己是小菜鸡。。。

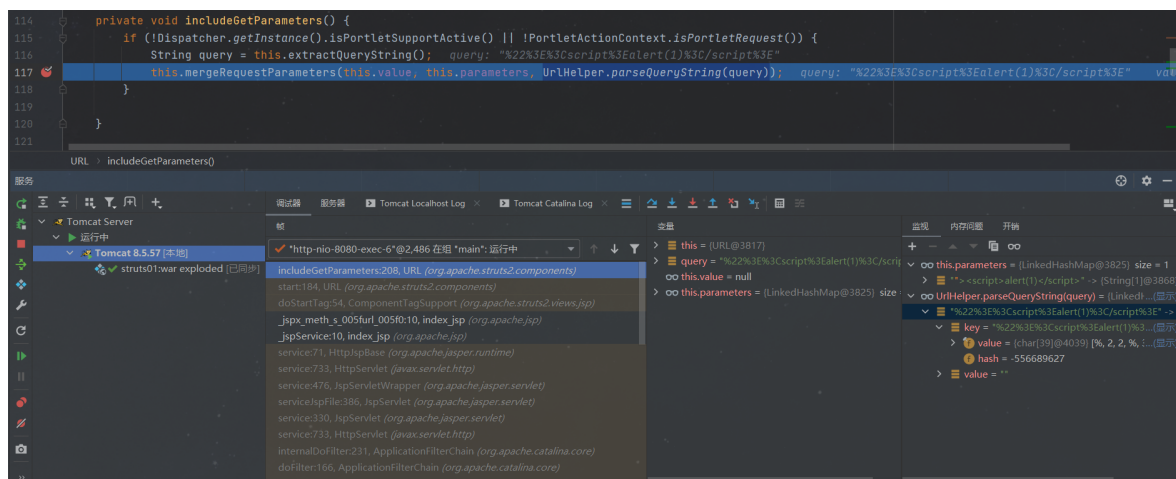
标签值为all的时候



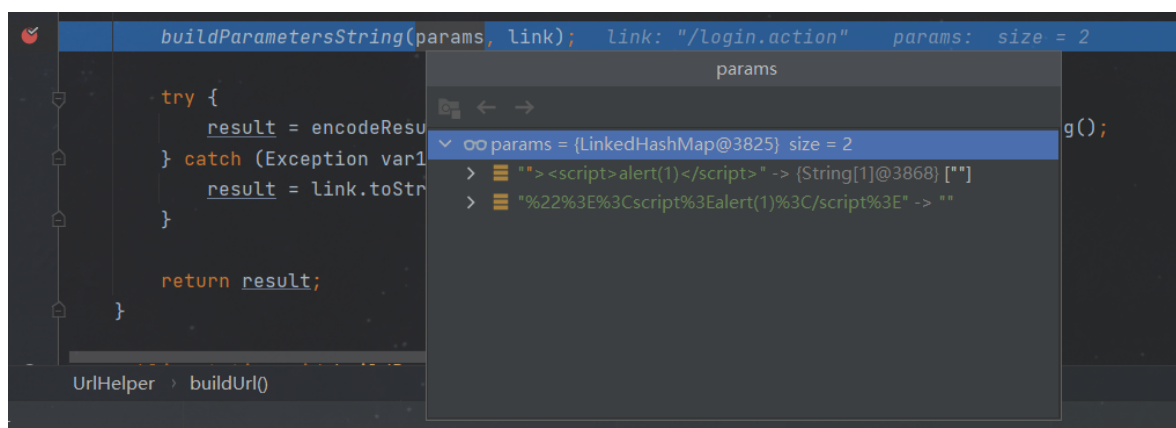
通过mergeRequestParameters方法添加参数



然后去调用includeGetParameters方法



最后 `URLHelper` 类处理 `this.parameters` 数据（将其拼接在URL中）并返回给用户



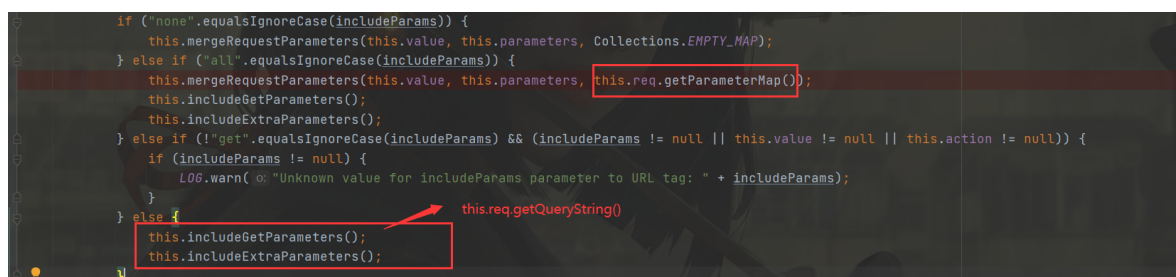
扩展

该漏洞是使用struts标签 `<s:url` 时候的问题并且 `includeParams="all"` 这样使我们在url上添加的payload进行处理之后返回给用户，形成xss。

主要原因在于:当 `includeParams=all` 时，会多执行一个 `mergeRequestParameters` 方法，而该方法会将 `this.req.getParameterMap()` 数据设置到 `this.parameters`。

如果 `includeParams=get`，那么 `this.parameters` 中的数据，仅是来自 `this.req.getQueryString()`

。



而 `this.req.getParameterMap()` 获得的数据会主动进行 `URLdecode`，但是 `this.req.getQueryString()` 不会。所以 `includeParams=get` 时，返回的数据是被 `URLEncode` 过的，因此不能触发 `XSS` 漏洞。

```
11 public class ParamsTest extends HttpServlet {
12     @Override
13     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
14         System.out.println("getQueryString():\t" + req.getQueryString());
15         Map<String, String[]> pm = req.getParameterMap();
16         for (String k: pm.keySet()){
17             System.out.println("getParameterMap():\t" + k);
18         }
19     }
20 }
21
```

Server Tomcat Localhost Log x Tomcat Catalina Log x

localhost:8081/struts2-001/p?<script>alert(1);</script>=hello

getQueryString(): %3Cscript%3Ealert();%3C/script%3E=hello

getParameterMap(): <script>alert();</script>

借mochazz师傅的图片

总结

简单的说就是使用上面的标签的时候，将我们输入我url值进行添加 `this.parameters` 然后在url上返回给用户(写到html中)形成xss。

漏洞修复

下图右边为官方修复后的代码（左图struts-core-2.0.8，右图为struts-core-2.0.11.1），可以明显看到修复代码仅仅是将 `<script>` 替换成 `script` 了。然而这种修复方式明显可以绕过，例如：`<script 1>alert(1)</script>`

```
# or Deliber.class (/home/mochazz/Desktop/java_helma/struts-2.0.8/1ib/struts2-core-2.0.8.jar)
105 106 if (result == null) {
107 108     result = (String)request.getAttribute("link");
109 110 }
111 112 if (result == null) {
112 113     result = request.getRequestURI();
113 114 }
114 115 link.append(result);
115 116 buildParametersString(params, link);
116 117 try {
117 118     result = encodeResult ? response.encodeURL(result) : result;
118 119 } catch (Exception var12) {
119 120     result = link.toString();
120 121 }

# or Deliber.class (/home/mochazz/Desktop/java_helma/struts-2.0.11.1/1ib/struts2-core-2.0.11.1/org/apache/struts2/views/s11)
126 127 link.append(result);
127 128 }
128 129 if (escapeAmp) {
129 130     buildParametersString(params, link);
130 131 } else {
131 132     buildParametersString(params, link, "&");
132 133 }
133 134 for(result = link.toString() result.indexOf("<script>") > 0 result = result.replaceAll("<script>", "script"));
134 135 }
135 136 try {
136 137     result = encodeResult ? response.encodeURL(result) : result;
137 138 } catch (Exception var13) {
138 139     result = link.toString();
139 140 }
```

参考

<https://dean2021.github.io/posts/s2-002/>

<https://mochazz.github.io/2020/06/17/java%E4%BB%A3%E7%A0%81%E5%AE%A1%E8%AE%A1%E4%B9%8BStruts2-002/#%E6%BC%8F%E6%B4%9E%E5%88%86%E6%9E%90>